
Project Title:

Identifying Public Schools Near Flood-Hazard Areas in the Texas Golden Triangle

Author: Michael Hawthorne

Share: Please do not share this project with future classes

Abstract:

The goal of this project is to perform a spatial analysis to determine what public schools are within a given proximity of FEMA flood hazard zones within Southeast Texas. While flooding is a more common occurrence globally, the Southeast Texas Golden Triangle area seems to be more prone to these types of events. With the changing global climate conditions, this exacerbates an already growing issue in this region of Texas. The local schools are no exception to this regional problem.

In my opinion, and in my lived experiences, meteorological events are seemingly becoming more problematic. Due to this, the need for an action plan is absolutely necessary in the event of a major storm or catastrophic event. The local roadways to and from the schools, and within these small communities can become inundated with flood waters very quickly. An actionable plan is necessary when it comes to getting your children to safety and in a timely manner. Knowledge and planning are the best ways to combat this issue.

With all of this information in mind, the goal of this project is to create a python script analysis to show how close these schools are, within a user-defined distance, to these FEMA flood zones. This script uses county boundary polygons, statewide FEMA Flood Hazard zones, a separate special flood hazard zone for Jefferson County, as well as a shapefile that contains all of the schools in the state of Texas. This script is not intended to show a risk-analysis of these schools, but rather to provide information on the proximity of local schools to FEMA flood Zones.

Introduction:

The research question that this project tries to analyze is to determine what local, public schools are within a user-defined distance of FEMA flood hazard zones within the Texas, Golden Triangle region. Having been raised in Southeast Texas in a little town in Jefferson County called Groves, this project really called me.

Since I was young, there have been several, major meteorological events that made living here quite difficult growing up and is even more challenging today for the residents that currently live there. I remember, when I was in the 4th grade at Groves Elementary, walking to the cafeteria during a particularly nasty storm. Unfortunately for the kids, the only way to get to the cafeteria was through an outdoor corridor. This corridor would very commonly flood. I remember students almost swimming to get to and from the cafeteria. When growing up, there were several hurricanes that my family had to run from. In 2017, Tropical Storm Harvey had me and my older brother tethered to each other and swimming for our lives to get to higher ground and to safety.

Even though these events seemed to be quite commonplace to me and my family then, the weather events of today seems to be getting steadily more challenging, and the flooding events also seem to be growing in proportion to this change. The hurricanes that occurred infrequently growing up, are now accompanied by flash flooding events that happen frequently in the local area. With these meteorological events being of more common occurrence now, the need for action and response, especially when it comes to caring for the needs of the local children, is of the highest importance. I know first-hand that the action and response of local government, officials, local-weather, news stations, first responders, and school related action committees can be lack-luster at the best of times and grossly negligent at the worst of times.

Planning is essential when it comes to having a timely response to these, potentially devastating meteorological events. I believe that this tool could help provide information to people and to help them realize how close their children's schools are to FEMA flood hazard zones. This is critical information, especially with the advent of these potentially deadly storms occurring more frequently than what individuals would like to think that they occur. I believe that with this information at hand, it could be a driving force for, at the very least, a more actionable plan from local officials, schools, and parents alike when it comes to flood preparation and action plans in the event of floods and major storm events.

Materials and Methods:

Study Area:

The general study area of this analysis involves the Southeast Texas Golden Triangle region. This area consists of three counties: Hardin, Jefferson, and Orange counties. These counties are centered along the Southeast Texas border between Texas and Louisiana. These counties contain a few larger communities that encompass the cities of Port Arthur, Beaumont, and Orange. This study area was chosen to provide a micro level look at the

particular spatial relationship between FEMA flood hazard zones and the local public schools that are within this selected community. The python script is engineered in a way that allows a user to select one of these three counties and to incorporate a distance buffer to determine which schools fall into this distance proximity.

Data:

The primary datasets that were used in this analysis consists of Texas public schools point data, FEMA flood hazard zones, and a Texas county boundary layer. The county boundary layer was used to pull the user-selected county information. The school location dataset was gathered from the Texas Education Agency 2024-2025 public data. This dataset contains point features showing the location and general school information of each of the data points. For this analysis, only the schools that have the category 'Regular Instructional' was used. This was to keep the analysis focused around regular, instructional public school institutions rather than alternative institutions.

The FEMA flood hazard polygons were used in this analysis to provide a spatial representation of where these real-world areas would be located. There were two FEMA flood hazard datasets used in this analysis. The first consists of a Statewide FEMA flood hazard dataset. This data contains the flood hazard zones for both Hardin and Orange counties. The second consists of a 2015 Preliminary FIRM flood-hazard data. Jefferson county flood hazard zones were gathered from this dataset. The Statewide dataset was incomplete and did not include Jefferson county. In the Flood Hazard zone datasets, only those areas that have the attribute of SFHA_TF (Special Flood Hazard Area) with a value of T (True) were used in this analysis. This identified which areas are considered Special Flood Hazard Areas.

The datasets used in this analysis had multiple different coordinate systems. During the analysis, all datasets are projected to NAD 1983 UTM Zone 15N to better provide a more common coordinate system between the datasets and to be able to perform the distance-based analysis that this python script is intended to analyze.

Workflow:

First Step:

The initial step of the workflow of this python script consists of an input from the user to select the county of choice. This step consists of both a script built on in a module and a script from the main notebook.

The module script looks like so:

```
#-----
# county selection: (while loop 1) (conditional for county and source)
def input_county_choice(): # defines the county input choice module

    valid_counties = ["Jefferson", "Hardin", "Orange"] # The valid 3 choices for counties
    # User input for county selection. .title() is a good way to keep the selection not case sensitive
    county = input("Please type either 'Jefferson', 'Hardin', or 'Orange' to choose your county: ").strip().title()
    # while loop for invalid county selections
    while county not in valid_counties:
        print("You have chosen an Invalid County Selection.")
        # prompts the user again for the proper county selection
        county = input("Please type either 'Jefferson', 'Hardin', or 'Orange': ").strip().title()
    # returns the valid county selection to the main script
    return county

# county selection continued:
def county_selection(county_layer, county_name, output_layer): # defines the county selection module
    # searches for the user designated county in the county layer
    county_search = f"CNTY_NM = '{county_name}'"
    # uses the select tool to choose the proper county information
    arcpy.analysis.Select(county_layer, output_layer, county_search)
    # returns county selection to the main script
    return output_layer
```

In this script, we define a function referred to as `input_county_choice`. This function allows for only 3 valid county selections. 'Jefferson', 'Hardin', and 'Orange' counties. This is run through a user defined input script. The input is then set to `.strip()` and `.title()` which makes the input not case sensitive and pulls off extra white space appropriately. A while loop is then used to determine whether the user defined input meets the valid 3 county criteria for this analysis. If the selection does not meet the criteria, then the script prompts the user to type a valid county name.

Then a `county_selection` function is defined. This function is intended to search for the user designated county within the county layer using the 'CNTY_NM' field in the county dataset to select the appropriate county of choice. This selection is then returned to the main notebook for analysis.

The main notebook script for county selection looks like so:

```
#-----
# county input:
county_choice = term_project_module.input_county_choice() # runs the county selection and user input through the module
term_project_module.county_selection(county_layer, county_choice, selected_county)

print(f"{county_choice} County has been chosen as the selection.", # prints the valid county selection name from the module
      "Loading next selection, please wait.") # asks the user to wait for the next selection (takes a while to load)

# county projection:
projected_county = output_gdb + r"/Selected_County_UTM" # sets the output path for the projected county output
# uses porjection layer module to project the user selected county
term_project_module.projection_layer(selected_county, projected_county)
```

This script calls the `input_county_choice` function from our module script and sets this information to a `county_choice` variable. The script then calls the `county_selection` function. Once the notebook has all of this information, the county choice is then printed

and the notebook states that the user selected county has been chosen as the selection and to wait as the program loads the next portion.

```
#-----  
# projection: (NAD 1983 UTM Zone 15N)  
def projection_layer(projection_input, projection_output): # defines the projection module  
    # sets the output coordinate system to a good spatial reference for Southeast Texas  
    coordinate_system = arcpy.SpatialReference("NAD 1983 UTM Zone 15N")  
    # uses the project tool to project the proper coordinate system  
    arcpy.management.Project(projection_input, projection_output, coordinate_system)  
    # returns the projection to the main script  
    return projection_output
```

While loading, the chosen county is then projected to NAD 1983 UTM Zone 15N using the defined projection_layer module from the module script. An output path for the projected layer is then created with the name 'Selected_County_UTM' for the output layer. This projection_layer module is used throughout the script to project all of the output layers.

Second Step:

The second step starts out with several new functions in the module script and a fair amount of script in the main notebook as well. Here is what the module script looks like for

this step:

```
#-----
# flood zone selection: (conditional statement)
def flood_zone(county_name, jefferson_fldzone, state_flood): # defines the flood_zone selection module
    # The jefferson flood zone information is separate from the State flood information. Jefferson county,
    # is not included in the regular, state flood zone dataset
    if county_name == "Jefferson": # special 'if' statement for Jefferson county
        flood_layer = jefferson_fldzone
    else:
        flood_layer = state_flood
    # sets the flood zone layer to state flood zone layer if Jefferson county is not selected
    # returns the proper flood zone layer to the main script
    return flood_layer

# flood zone clipping:
# This will clip the state flood polygons to the county of choice (Hardin, Orange)
def flood_zone_clip(county_name, flood_layer, county_layer, output_layer):
    # defines flood zone clip module
    # no reason to clip if the selection choice is 'Jefferson county'
    if county_name == "Jefferson":
        # returns the selection to the main notebook
        return flood_layer
    # clip the flood zones from the state layer for Hardin and Orange counties
    else:
        arcpy.analysis.Clip(flood_layer, county_layer, output_layer)
        # returns the selection to the main notebook
        return output_layer

# Selects only those areas that are considered FEMA Special Flood Hazard Areas
def sfha_selection(flood_layer, output_layer):
    # selects only those polygons that are set to 'True' for Special Flood Hazard Areas
    sfha_search = "SFHA_TF = 'T'"
    # uses the analysis select tool to select those SFHA polygons
    arcpy.analysis.Select(flood_layer, output_layer, sfha_search)
    # returns the output selection to the main notebook
    return output_layer

# flood zone dissolve module:
# defines the dissolve flood zone module
def dissolve_flood_zone(input_layer, output_layer):
    # uses the dissolve tool to dissolve the selected SFHA polygons
    arcpy.management.Dissolve(input_layer, output_layer)
    # returns dissolved selection to main notebook
    return output_layer
```

This selection defines a number of functions including `flood_zone`, `flood_zone_clip`, `sfha_selection`, and `dissolve_flood_zone`.

The `flood_zone` function consists of a conditional statement that determines whether the user-defined county is either Jefferson county or one of the two other counties. The reason for this is because the Jefferson county flood zone data is in a completely different dataset than the statewide dataset that both Orange and Hardin county flood zone information is located. The if/else statement either finds that a user has chosen 'Jefferson' county to use the `Jefferson_fldzone` layer or uses the `state_flood` dataset instead. This information is then returned to the notebook.

`Flood_zone_clip` function uses an additional conditional statement. Dependent on whether a user has chosen Jefferson county determines whether this module will need to be used. If the user chooses either Hardin or Orange counties then the conditional statement is set up to use the clip analysis tool to clip the selected flood zone from the state flood zone layer. If the user chooses Jefferson county, no clip is needed. The Jefferson county flood

zone data is in a standalone format. After the clip analysis, the chosen output is then returned to the main notebook.

The next function defined is 'sfha_selection'. This function defines sfha_search variable as 'SFHA_TF = 'T'. This variable is then used in the select analysis tool to only select the flood zone polygons that have an SFHA_TF value of 'T'. SFHA stands for Special Flood Hazard Area. The 'T' stands for True. The select analysis tool will only select those polygons that meet the specified criteria. This output is then returned to the notebook.

The last function in this section is the dissolve_flood_zone function. This function uses the dissolve tool to dissolve the selected SFHA polygons into a single output. This is then returned to the notebook as well.

We will now look at the script used in the main notebook for flood-zone selection:

```
#-----
# flood zone choice: # If Jefferson county is chosen, the jefferson_fldzone will be used, if not, then state_flood will be used
selected_flood_zone = term_project_module.flood_zone(county_choice, jefferson_fldzone, state_flood)
# creates the new county flood zone output and sends it to the output gdb under the name County_Flood_Zone
county_flood_selection = output_gdb + r"/County_Flood_Zone"
# call the function flood zone clip function
county_flood = term_project_module.flood_zone_clip(county_choice, selected_flood_zone, projected_county, county_flood_selection)

# projects the county flood areas to a proper spatial reference for Southeast Texas:
projected_flood = output_gdb + r"/County_Flood_Zones_UTM"
# uses the projection layer module to project the selected county flood areas
term_project_module.projection_layer(county_flood, projected_flood)

# call special flood hazard area selection module and sets the output name to Selected_SFHA:
selected_sfha = output_gdb + r"/Selected_SFHA"
# calls the special flood hazard selection module
term_project_module.sfha_selection(projected_flood, selected_sfha)

# calls the dissolve flood hazard area function from module and sets the output name to Dissolved_SFHA:
dissolved_sfha = output_gdb + r"/Dissolved_SFHA"
# dissolves all SFHA polygons that passed the initial selection using the dissolve flood zone module
term_project_module.dissolve_flood_zone(selected_sfha, dissolved_sfha)
```

This script starts off calling the flood_zone function in the module script and defines it as the selected_flood_zone variable. This county flood selection is then output as the specified layer and sent to the output gdb for Hardin and Orange county selections. The flood_zone_clip module is then called and set as a the appropriate variable name.

This information is then run through the projection_layer function and the projected layer is named and sent to the output gdb.

Once the projected layer is run, the script then calls the sfha_selection function and sends those results to the output gdb.

The same is done for the dissolved_sfha layer. The function is called and the output is then appropriately named and output to the gdb.

All of the output layers are also defined as separate variables in this process and named appropriately.

Third Step:

The third step also consists of a script for both the module and the main notebook. The script for the module looks like so:

```
#-----
# flood buffer/proximity: (while loop 2) (conditional for buffer)
# defines the buffer distance module
def input_buffer_distance():

    # prompts the user to input a buffer distance as a float:
    buffer_distance = float(input(
        "Enter a buffer distance number (in miles) from the flood zone: "
        "Entering a value of '0' will include all schools that are directly in a flood zone: "))
    # makes it to where you can just select all schools that are just the selected flood zone area
    # while loop to prevent negative input values:
    while buffer_distance < 0:
        print("Input buffer distance cannot be a negative distance.")
        # prompts the user again to enter either 0 or a positive number as a float:
        buffer_distance = float(input(
            "Enter either '0' or a number greater than 0 for the buffer distance selection: "))
    print("Please wait on your selection output.")
    # because the prompts can be a little slow to return, tells the user to wait patiently for next prompt

    # returns the buffer distance value to the notebook
    return buffer_distance

# buffer module:
# defines the actual flood buffer module (not just the distance input)
def flood_buffer(flood_layer, buffer_distance, output_layer):

    # If the buffer selection is greater than 0, then there is a buffer output:
    if buffer_distance > 0:
        # sets the buffer distance to miles
        distance = f"{buffer_distance} Miles"
        # runs the buffer analysis tool from user input information
        arcpy.analysis.Buffer(flood_layer, output_layer, distance)

        # returns the buffer output to the notebook
        return output_layer
    # When '0' is entered, this selects the flood zone polygons to use for analysis instead
    else:
        return flood_layer
```

The script for this section consists of a few additional functions that are intended to get a user-defined buffer distance and to then create an actual buffer using the Buffer analysis tool.

An input_buffer_distance function is defined. This module asks for an input 'float' value that is used as the user-defined buffer distance. A while loop is used so that the input value cannot be a negative number. In this selection, a user can select '0' which would then allow the script to only select those schools that are within the actual SFHA polygons to be selected by the analysis. Once the user sets the buffer distance, this information is returned to the notebook.

A flood_buffer function is then defined. This function uses a conditional to determine whether a user entered either 0 or a positive number. If a user selects a positive number then this number is then set as the distance in miles and is run through the buffer analysis tool. If the user enters 0, then only the flood zone polygon is used in the analysis. This selection is then returned to the notebook.

The main notebook for this section only consists of a few lines of code:

```
#-----  
# buffer zone choice:  
# asks for user input for buffer zone distance from buffer module:  
buffer_distance = term_project_module.input_buffer_distance()  
  
# creates the buffer output path and names the output Buffered_SFHA:  
buffered_sfha = output_gdb + r"/Buffered_SFHA"  
  
# will either return the original flood zone or create the valid buffer using the user defined distance:  
flood_analysis_area = term_project_module.flood_buffer(dissolved_sfha, buffer_distance, buffered_sfha)
```

This section essentially calls the input_buffer_distance function and the flood_buffer function. The output paths are defined and named appropriately.

Fourth Step:

The fourth step consists of most of the rest of the scripts. This section also includes both a module script and a main notebook script. The module script looks like so:

```

#-----
# school selection:
# module that selects regular, instructional schools in the county of choice
def school_selection(school_layer, county_name, output_layer):
    # sets the county value to the appropriate county chosen
    county_value = f"{county_name} County"
    # chooses the schools based on the county and instruction type in the school layer
    school_search = (f"Subregion = '{county_value}' AND "f"USER_instr = 'REGULAR INSTRUCTIONAL'")
    # uses the select analysis tool to select the schools that meet this criteria
    arcpy.analysis.Select(school_layer, output_layer, school_search)
    # returns the chosen schools to the notebook
    return output_layer

#-----
# determine at risk schools: (selects all schools that fall within the user specified criteria)
# defines the at risk schools module
def at_risk_schools(school_layer, flood_analysis_area, output_layer):

    # makes a school feature layer for the analysis
    arcpy.management.MakeFeatureLayer(school_layer, "school_layer")

    # uses the select layer by location tool and 'intersect' as the relationship value
    arcpy.management.SelectLayerByLocation("school_layer", "INTERSECT", flood_analysis_area)

    # saves the selected at-risk schools in a new layer using 'CopyFeatures'
    arcpy.management.CopyFeatures("school_layer", output_layer)

    # returns the selected schools to the notebook
    return output_layer

# determines the count of at-risk schools: (just to get a number output of the selected schools)
# defines the school count module
def school_count(school_layer):

    # uses 'GetCount' to get the number of selected schools
    count = int(arcpy.management.GetCount(school_layer)[0])

    # returns the count value to the notebook
    return count

```

This section defines several functions as well. This includes a `school_selection` function, an `at_risk_schools` function, and a `school_count` function.

The `school_selection` function is defined. This function selects schools based on the user defined county using the 'county_value' unique value of the 'Subregion' field and the 'REGULAR INSTRUCTIONAL' unique value of the 'USER_instr' field. The select analysis tool is then used to select those schools that meet these two criteria. This selects the schools that are within the county of choice and those that meet the criteria of 'REGULAR INSTRUCTIONAL'.

An `at_risk_schools` function is then defined. This function does a number of different things. The first is to use the `MakeFeatureLayer` tool to create a temporary `school_layer` from the projected school feature class. This temporary layer is then used to perform the analysis. The `selectLayerByLocation` management tool is then run on the school points layer, using 'Intersect' as the relationship value on the `flood_analysis_area`. Those values are then copied to an output layer using the `CopyFeatures` tool. This information is then returned to the notebook.

The last function defines a school_count function. This uses the GetCount management tool to count the number of schools that are selected by the entire analysis. This count is set as an integer. The count is then returned to the notebook.

The last section goes over the rest of what is written in the main notebook:

```
#-----
# results output:
# creates the output path for the schools selection and names the output Selected_Schools:
selected_schools = output_gdb + r"/Selected_Schools"

# calls the school selection function from module
term_project_module.school_selection(school_layer, county_choice, selected_schools)

# projects selected schools to a proper projection for Southeast Texas
projected_schools = output_gdb + r"/Selected_Schools_UTM"
# calls the projection layer module
term_project_module.projection_layer(selected_schools, projected_schools)

# creates the at-risk schools selection output path and names output At_Risk_Schools:
at_risk_school_output = output_gdb + r"/At_Risk_Schools"

# calls the at-risk schools function from module:
at_risk_schools = term_project_module.at_risk_schools(projected_schools, flood_analysis_area, at_risk_school_output)

# calls the school count module for the count of schools selected in analysis
number_at_risk = term_project_module.school_count(at_risk_schools)

# prints the number of at-risk schools selected through analysis:
print(f"{number_at_risk} is the number of schools that are within the specified flood hazard buffer area.",
      "All created layers have been saved to the specified gdb Output path location.")

# uses a Search Cursor to browse and then print the specified school names that are within the analysis:
with arcpy.da.SearchCursor(at_risk_school_output, ["USER_Sch_1"]) as cursor: # uses a cursor to search for School Name
    print("Here is a list of all of the schools in the specified criteria: ") # prints a statement for the specified header
    print("School Name:") # added a School Name heading for the printout
    print("-----") # adds an additional space to separate header from the output data
    for row in cursor: # uses a 'for' loop to print out row-24 (USER_Sch_1) for the at_risk_school_output shapefile
        print(row[0])
```

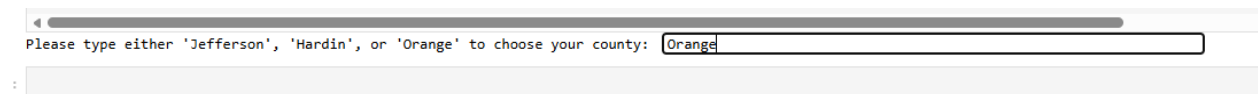
Ultimately, this is the results output section. The initial script here defines selected_schools variable and the name and output path for the appropriate variable. The school_selection function is then called. The projected_schools' variable is defined and the name and output path for this variable as 'Selected_Schools_UTM'. The projection_layer function is then called to project the selected schools to NAD 1983 UTM Zone 15N. Afterwards an at_risk_school_output variable is then set and the proper output path and name is set. The at_risk_schools function is then called and set equal to the at_risk_schools variable. The school_count function is then called and set equal to the 'number_at_risk' variable. This variable is then printed to specify the number of schools that meet the user-specified criteria.

The last section uses a SearchCursor to iterate through all the school names meets the analysis criterion and then a loop is used to print those school names out individually.

Throughout the entire analysis, multiple output layers are created and can therefore be used as a visible representation for future use as well as the actual count of schools and the list that is also then output at the end of the analysis.

Results:

The results section will go over a test of the overall script itself. This test will use 'Orange' county as the initial county selection and a fairly small distance of .15 miles for the buffer distance selection. Here is the initial input question for the user. 'Orange' will then be typed in this section.



```
Please type either 'Jefferson', 'Hardin', or 'Orange' to choose your county: Orange
```

A short loading time goes by and the second prompt then asks the user to input a buffer distance in miles from the flood zone. The value of .15 will then be entered in this input field.

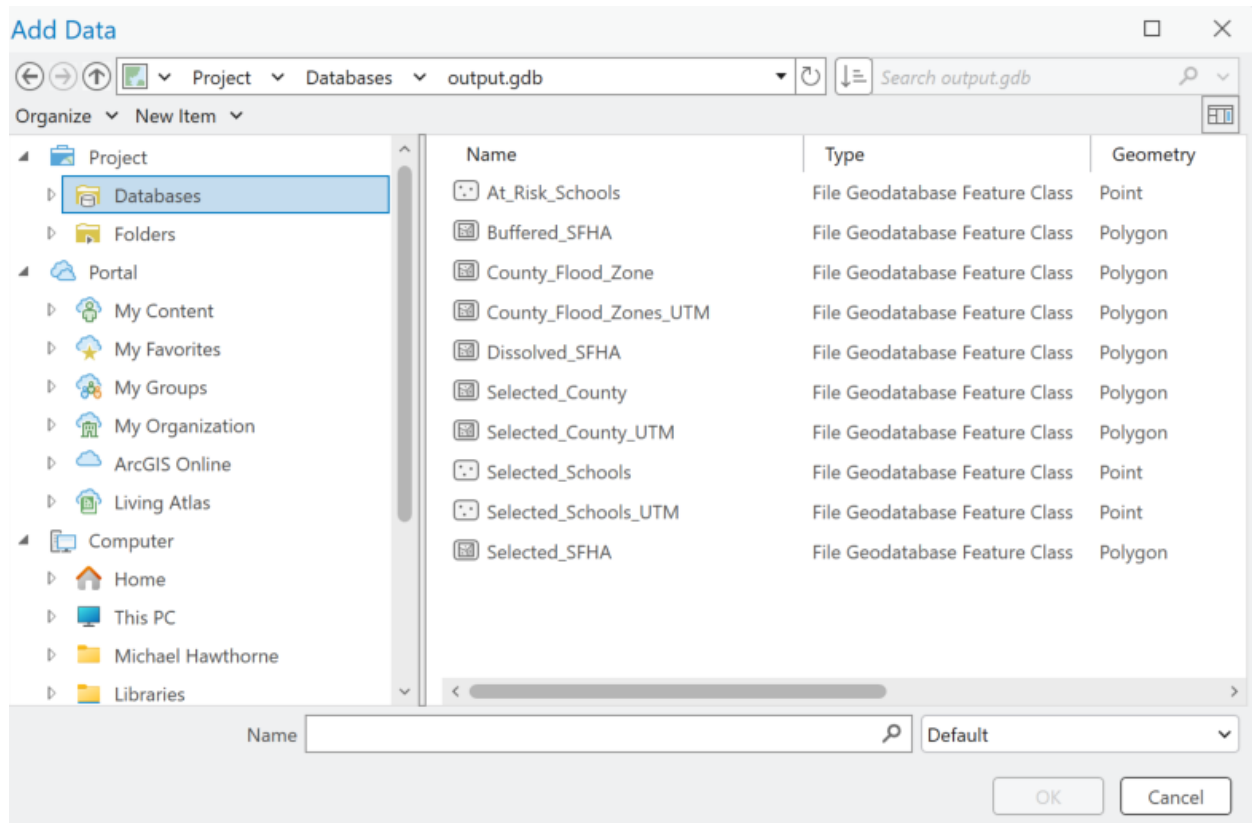
```
Please type either 'Jefferson', 'Hardin', or 'Orange' to choose your county: Orange
Orange County has been chosen as the selection. Loading next selection, please wait.
Enter a buffer distance number (in miles) from the flood zone: Entering a value of '0' will include all schools that are directly in a flood zone:
.15
```

Once the script fully runs the complete output of the script is then complete. This is the full prompts, inputs, and outputs that the script will produce once everything has run appropriately.

```
Please type either 'Jefferson', 'Hardin', or 'Orange' to choose your county: Orange
Orange County has been chosen as the selection. Loading next selection, please wait.
Enter a buffer distance number (in miles) from the flood zone: Entering a value of '0' will include all schools that are directly in a flood zone: .15
Please wait on your selection output.
16 is the number of schools that are within the specified flood hazard buffer area.
All created layers have been saved to the specified gdb Output path location.
Here is a list of all of the schools in the specified criteria:
School Name:
-----
TEKOA ACADEMY OF ACCELERATED STUDIES - ORANGE
BRIDGE CITY H S
BRIDGE CITY MIDDLE
BRIDGE CITY EL
BRIDGE CITY INT
ORANGEFIELD H S
WEST ORANGE-STARK H S
ORANGEFIELD J H
ORANGEFIELD EL
M B NORTH E C LRN CTR
WEST ORANGE-STARK MIDDLE
PINE FOREST EL
OAK FOREST EL
VIDOR MIDDLE
LITTLE CYPRESS J H
LITTLE CYPRESS EL
```

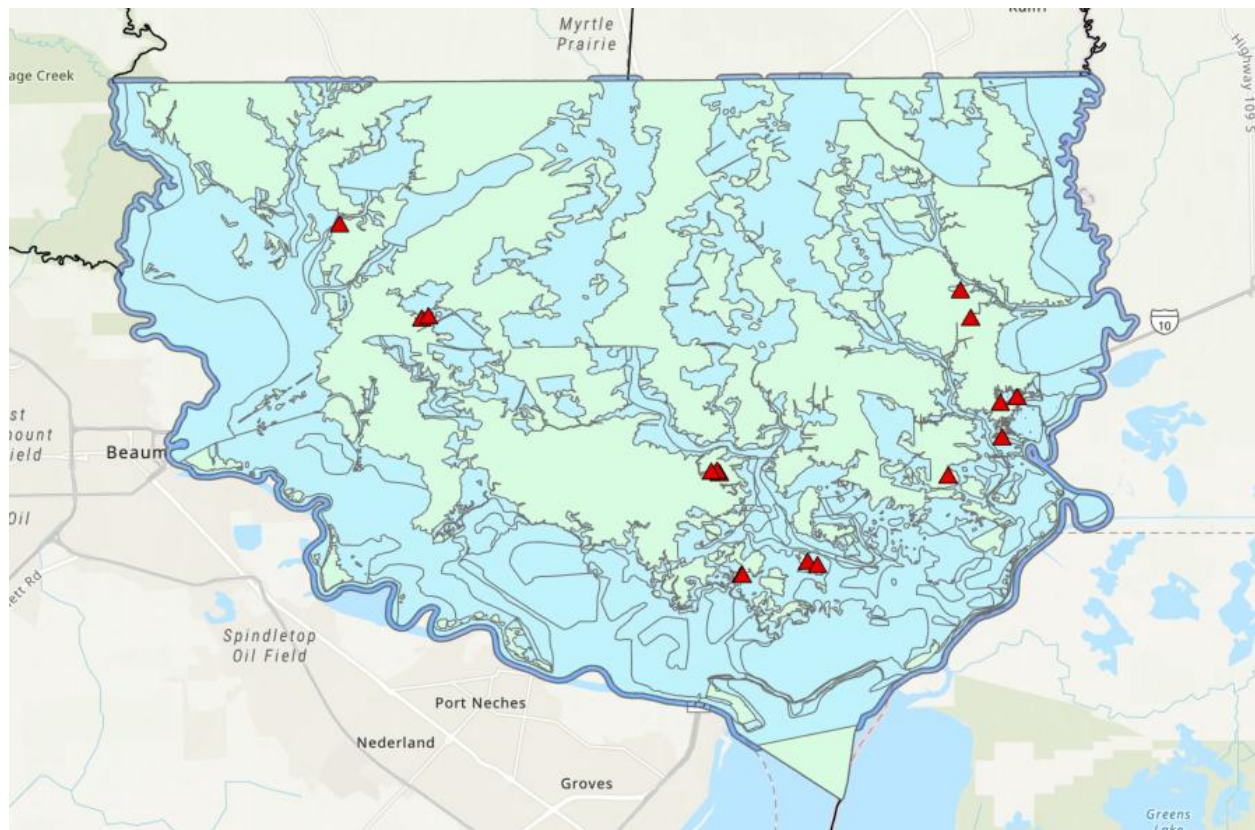
Notice that the script produced a count of 16 schools that meet the overall user-defined analysis. A note stating that all created layers have also been saved to a particular user-defined output gdb location. These output layers can then be used to view the analysis results spatially. Lastly, a list of all of the schools that meet the criteria has also been printed.

When looking at the output layers that are produced by the script in Arc, you will notice a number of layers that were created in this process.



This layer list contains the selection output layers, as well as the dissolved layers, and a Point feature class of the schools that were ultimately selected by the analysis, referred to as: At_Risk_Schools. Even though the layer is referred to as At_Risk_Schools, risk analysis was not used in this process.

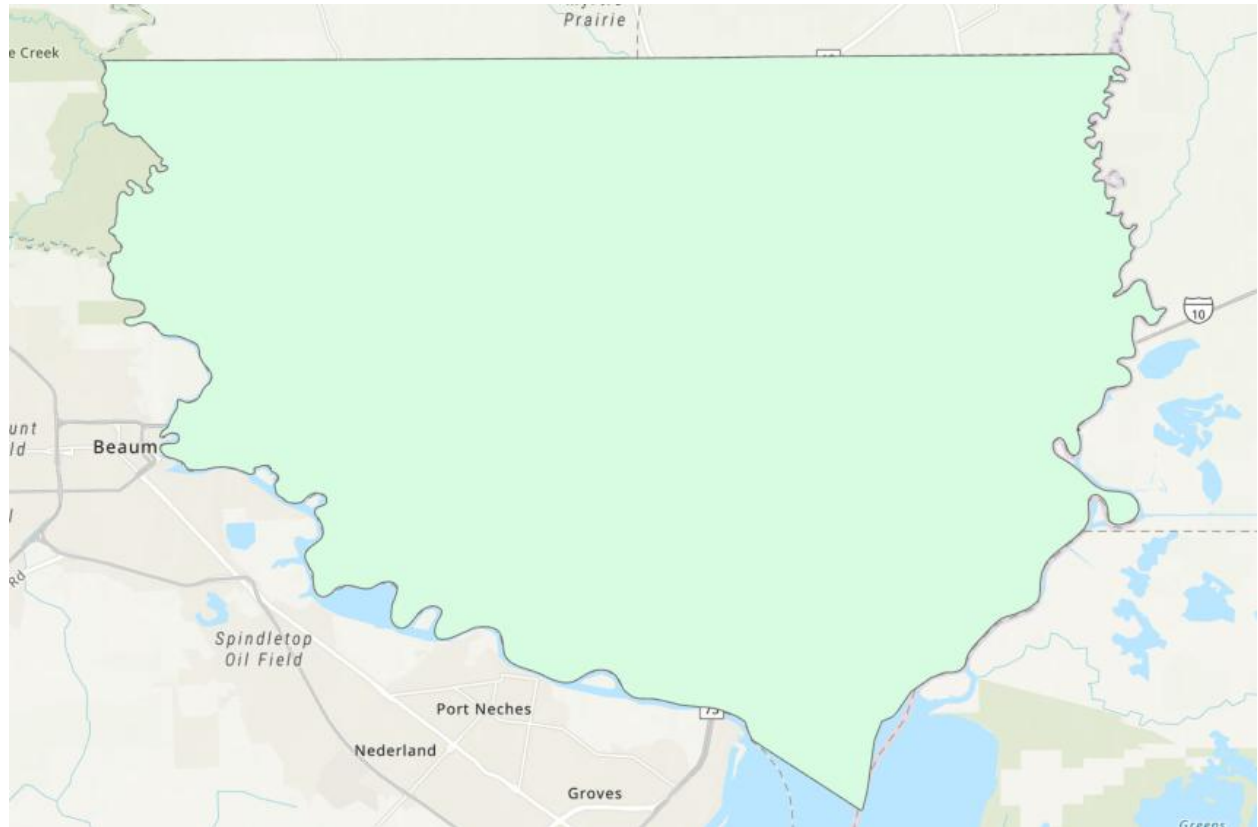
Once all the layers are uploaded to ArcPro, they should appear visually as so:



In this image, both the 'Selected_Schools' and 'Selected_Schools_UTM' layers are turned off, and the symbology of the point features are set to red triangles to showcase the locations.

We will now look at each output layer one-by-one in context of when they were output in the script.

The first output layer is Selected_County_UTM:



This output layer essentially just shows the county of choice, but had been projected to the proper spatial extent.

The next two outputs are the 'County_Flood_Zone' and the 'County_Flood_Zones_UTM' these outputs are essentially the same, however, the second one has been projected appropriately. This figure showcases the projected layer:

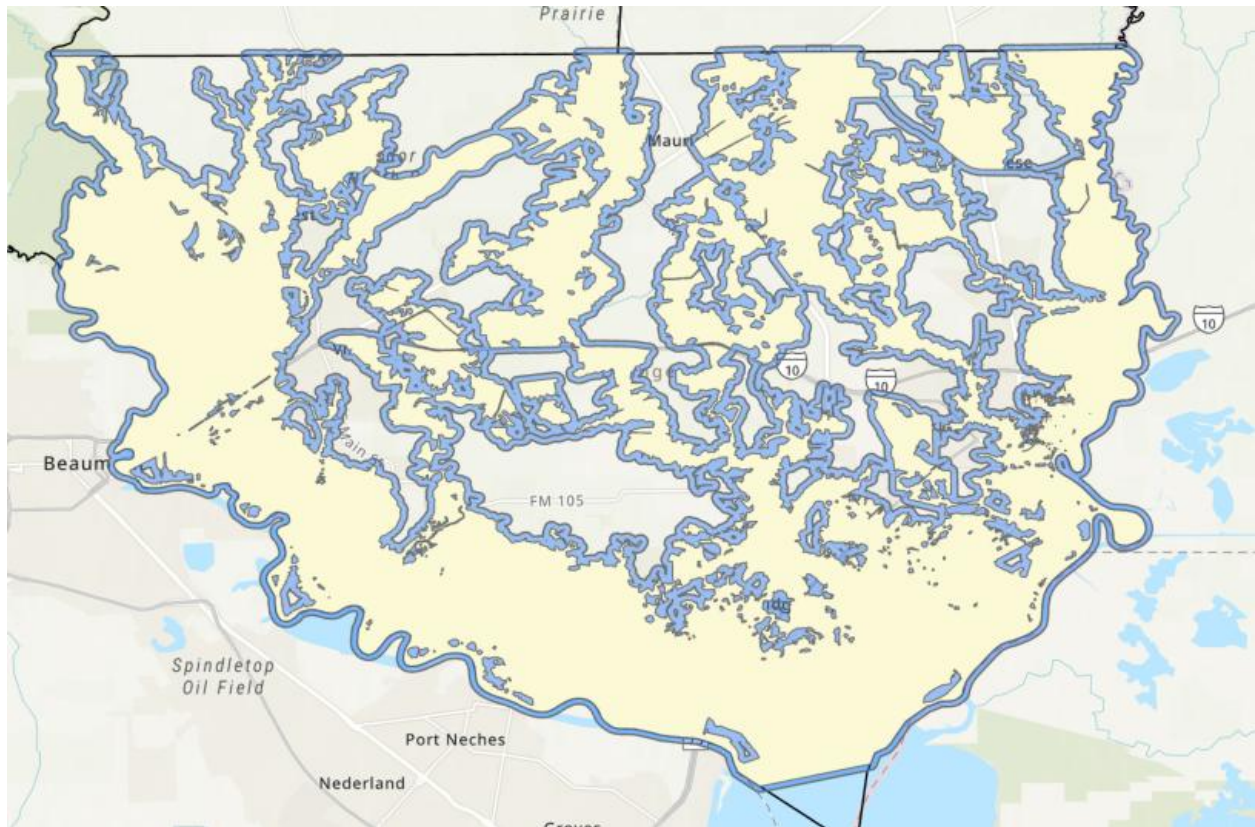


The next two figures showcase the 'Selected_SFHA' and the 'Dissolved_SFHA' layers:

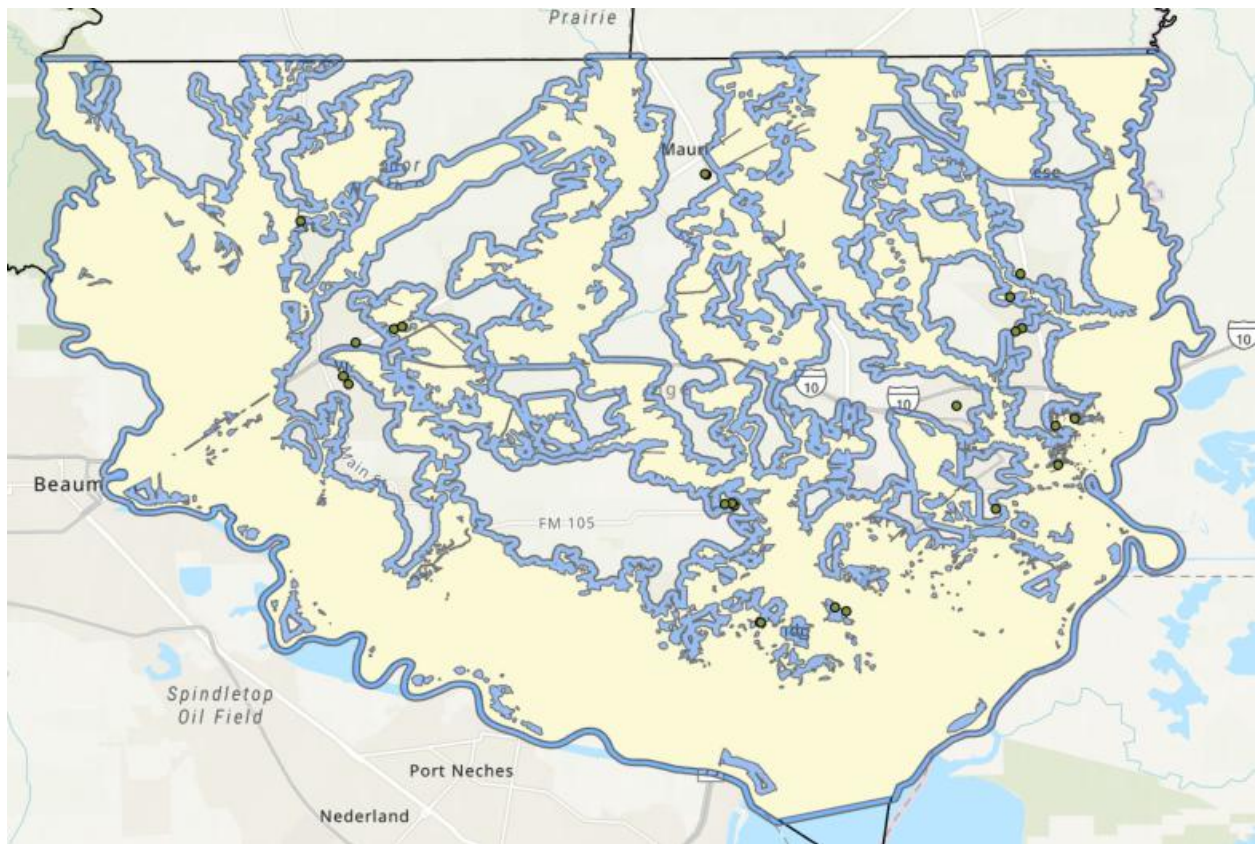
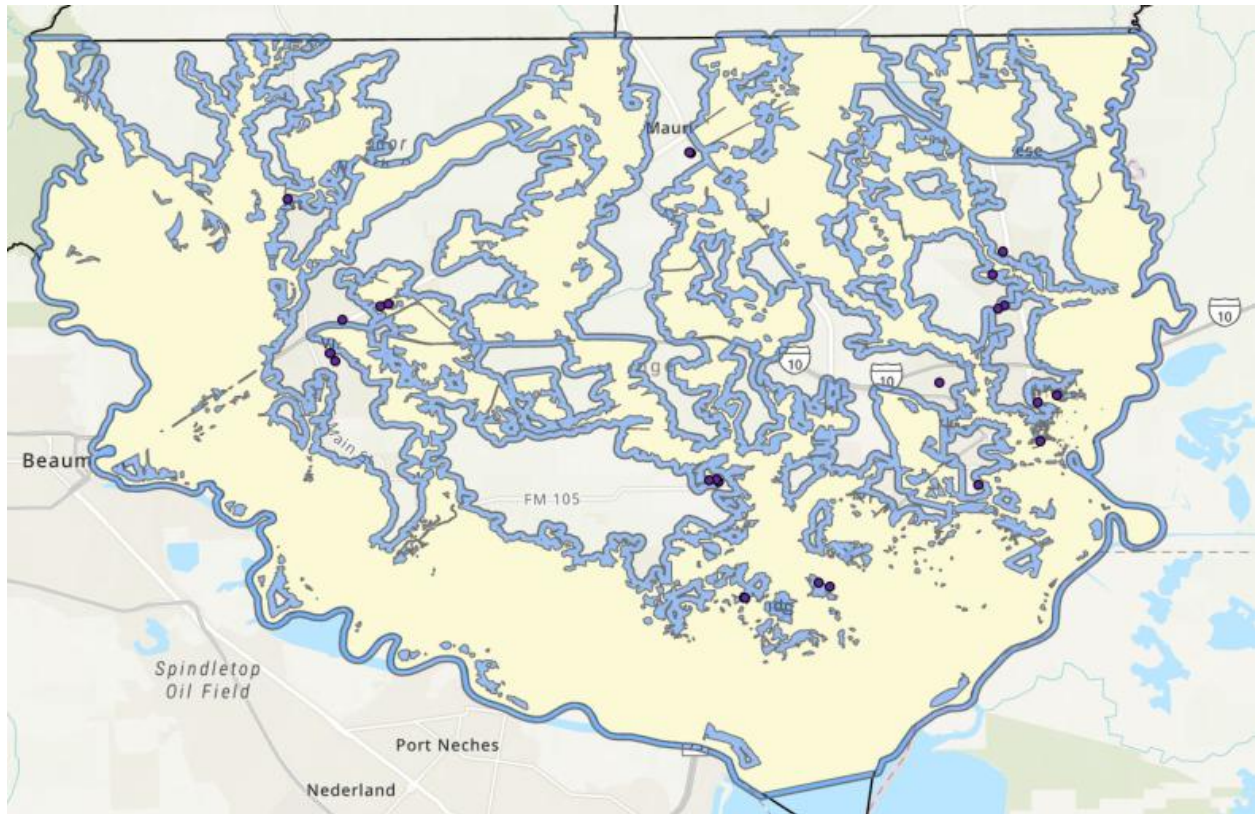


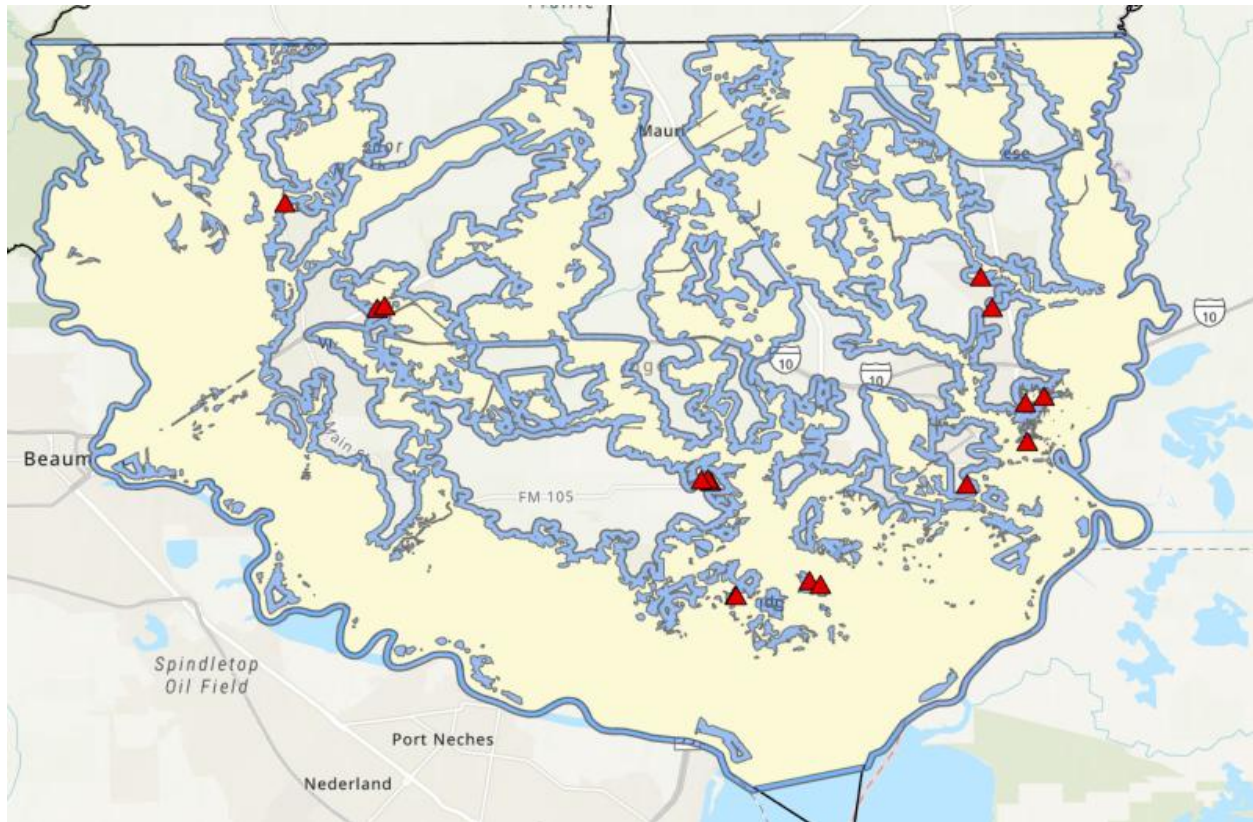
You will notice that the dissolved layer has essentially taken the initial SFHA layer and dissolved all of the SFHA polygons into one useable polygon for the analysis.

The 'Buffered_SFHA' layer, when used in conjunction with the 'Dissolved_SFHA' boundary gives us a better understanding of what our analysis actually looks like before we incorporate the final addition of the selected schools point features.



The next three layers are the output schools layers. The 'Selected_Schools' layer, the 'Selected_Schools_UTM' layer (which is the projected school locations), and the 'At_Risk_Schools' layer (which is the ultimate schools selection from our analysis).





These output layers and the results listed on the script output itself is the ultimate outcome from running the script when selecting 'Orange' county and a buffer zone of '.15' miles. A count of 16 schools meet the overall analysis and those school names are listed in the results.

Discussion and Conclusions:

The overall result of the analysis is clear. Out of the 23 schools in Orange County that meet the criteria of 'REGULAR INSTRUCTIONAL', 16 of these are either in a FEMA Flood-Hazard Area or within a range of .15 miles of FEMA Flood-Hazard zones. This is a clear indicator, at least, that the flood zones in Orange County encompass most of the schools, or are within less than a mile of those public institutions. With this information in mind, especially in the 'Orange' county area that this analysis intends to study, there is a clear need for an action plan when it comes to flooding and making sure that the children that go to these schools are in safe hands when these events occur. The proximity of these Flood-Hazard zones is much closer than they truly appear to be and this analysis proves that in 'Orange' county, Texas, that truly seems to be the case when it comes to our public education institutions. With the advent of analyses like these, there is a driver here for change. That can especially be the case for areas such as the Texas, Golden Triangle area. These low lying areas near the gulf coast can be incredibly susceptible to localized

meteorological events that could have a major and devastating effects on the people who reside there for you years to come. The overall intention of this analysis was to verify how close the public school institutions truly are to some of these FEMA Flood-Hazard Zones and to potentially garner attention and ultimately, the need to incorporate more efficient and actionable flood action plans that could help assuage issues that ultimately will occur.

This python script is fairly modular and can be modified to run analyses on other counties in the State of Texas. However, there are some limitations and controlling factors that may come into play. In some counties, FEMA Flood-Hazard Areas may not be mapped out accordingly or the data is missing. In those instances, FEMA may have Preliminary FIRM flood-hazard data that could be used in the instances where the State-wide data may be missing or incomplete. This script can also be used in other states throughout the United States. The python paths and input data would have to be updated accordingly to those datasets and the input values updated on the module to accommodate for these changes. Additional data can also be used and incorporated if more accurate information becomes available in the future. The addition of private meteorological data could be used in conjunction with public and governmental datasets to make a more lasting, future impact on these communities.

References:

Federal Emergency Management Agency. (2015). Jefferson County, Texas, revised preliminary Flood Insurance Rate Map (FIRM) database [Data set]. FEMA Map Service Center. <https://msc.fema.gov/>

Federal Emergency Management Agency. (2026). National Flood Hazard Layer (NFHL): Texas [Data set]. FEMA Map Service Center. <https://www.fema.gov/flood-maps/products-tools>

Texas Department of Transportation. (n.d.). Texas county boundaries [Feature layer]. ArcGIS Online.
https://services.arcgis.com/KTcxiTD9dsQw4r7Z/arcgis/rest/services/Texas_County_Boundaries/FeatureServer

Texas Education Agency. (n.d.). Schools 2024 to 2025 [Feature layer]. ArcGIS Online.
https://schoolsdata2-tea-texas.opendata.arcgis.com/datasets/80c162b9008a40d681c127874722670f_0/about

Federal Emergency Management Agency. (2025). Available flood hazard information—State of Texas [PDF]. U.S. Department of Homeland Security.
https://www.fema.gov/sites/default/files/documents/20250131_AvailableData_TX.pdf